

Algorithm2: *Problem Solving* with *Algorithms* and *Data Structures*

Lecture 03.

분할 정복

경북대학교 배준현 교수
(joonion@knu.ac.kr)



분할 정복: *Divide-and-Conquer*

가장 유명한 알고리즘 디자인 패러다임: 각개 격파

(*Divide*) 주어진 문제를 **둘 이상의 부분 문제**로 나눈 뒤

(*Conquer*) 각 **부분 문제에 대한 답을 재귀 호출**을 이용해 계산하고

(*Combine*) 각 부분 문제의 답으로부터 **전체 문제의 답**을 계산

재귀 호출과 다른 점:

문제를 **한 조각과 나머지 전체**로 나누는 대신 **거의 같은 크기의 부분 문제**로 나눈다.

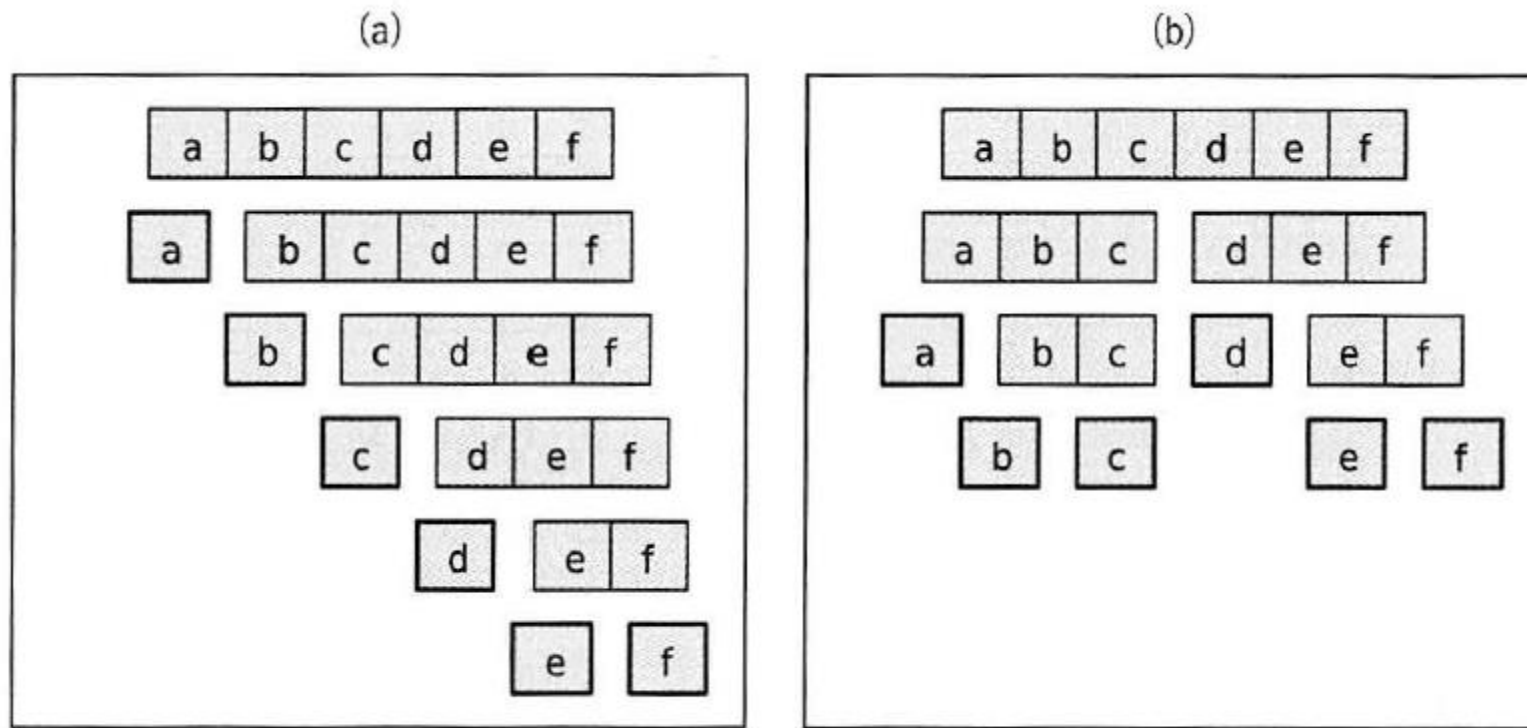


그림 7.1 일반적 재귀 호출과 분할 정복의 차이



자연수의 합 문제: 1부터 n 까지의 합 $s = 1 + 2 + \dots + n$ 구하기

일반적인 재귀 호출 알고리즘:

n 개의 숫자의 합을 구하는 작업을

n 개의 조각으로 쪼개어,

이 조각 중 하나를 떼내어 해결하고

나머지 조각들을 재귀 호출로 해결

```
def rsum(n):  
    if n == 1:  
        return 1  
    else:  
        return n + rsum(n - 1)
```



자연수의 합 문제: 1부터 n 까지의 합 $s = 1 + 2 + \dots + n$ 구하기

분할정복 알고리즘:

n 개의 숫자의 합을 구하는 작업을

n 개의 조각으로 쪼개어,

이들을 반으로 똑 잘라 (n 을 짝수라고 가정)

$\frac{n}{2}$ 개의 조각들로 만들어진 부분 문제 2개를 만든다.

$$s = 1 + 2 + \dots + n$$

$$= \left(1 + 2 + \dots + \frac{n}{2}\right) + \left(\left(\frac{n}{2} + 1\right) + \dots + n\right)$$

부분 문제를 재귀적으로 풀기 위해서

두 번째 부분 문제를 원래 문제와 같은 꼴로 표현

$$\begin{aligned} \left(\left(\frac{n}{2} + 1\right) + \dots + n\right) &= \left(\left(\frac{n}{2} + 1\right) + \dots + \left(\frac{n}{2} + \frac{n}{2}\right)\right) \\ &= \frac{n}{2} \times \frac{n}{2} + \left(1 + 2 + \dots + \frac{n}{2}\right) \end{aligned}$$



Lec. 03. 분할 정복

$$\begin{aligned}
 \text{sum}(n) &= 1 + 2 + \dots + n \\
 &= \left(1 + 2 + \dots + \frac{n}{2}\right) + \left(\left(\frac{n}{2} + 1\right) + \dots + n\right) \\
 &= \text{sum}\left(\frac{n}{2}\right) + \frac{n}{2} \times \frac{n}{2} + \text{sum}\left(\frac{n}{2}\right) \\
 &= 2 \times \text{sum}\left(\frac{n}{2}\right) + \left(\frac{n}{2}\right)^2
 \end{aligned}$$

위의 분할은 n 이 짝수인 경우이기 때문에

n 이 홀수인 경우:

짝수인 $(n - 1)$ 까지의 합을 계산(재귀 호출)하고
 n 을 더한다.

```
def fsum(n):
    if n == 1:
        return 1
    else:
        if n % 2 == 1: # 홀수이면
            return fsum(n - 1) + n
        else: # 짝수면
            return 2 * fsum(n // 2) + n * n // 4
```



분할정복 알고리즘의 시간 복잡도 분석

두 함수 모두 내부에 반복문이 없으므로, 시간 복잡도는 함수 호출 횟수에 비례

$rsum()$: 함수를 호출할 때마다 문제의 크기가 1 줄어들기 때문에 모두 n 번의 함수 호출이 필요

$fsum()$: 함수를 호출할 때마다 최소한 두 번에 한 번 꼴로 n 이 절반으로 줄어든다.

따라서, $fsum()$ 의 호출 횟수는 $rsum()$ 의 호출 횟수보다 훨씬 적을 것이다!



Lec. 03. 분할 정복

$fsum(11)$ 을 실행할 때 재귀 호출의 입력이 어떻게 변화하는지를 **이진수 표현**으로 살펴보자.

$$fsum(101\textcolor{red}{1}_2) = fsum(1010_2) + 11$$

$$fsum(101\textcolor{red}{0}_2) = 2 \times fsum(101_2) + 25$$

$$fsum(10\textcolor{red}{1}_2) = fsum(100_2) + 5$$

$$fsum(100\textcolor{red}{2}) = 2 \times fsum(10_2) + 4$$

$$fsum(1\textcolor{red}{0}_2) = 2 \times fsum(1_2) + 1$$

$$fsum(1_2) = 1$$

재귀 호출을 할 때 n 의 이진수 표현의
마지막 자리가 **1**이면 **0**으로 바뀌고,
마지막 자리가 **0**이면 끝자리가 없어진다.

$fsum()$ 의 총 호출 횟수는

n 의 이진수 자릿수 + 첫 자리를 제외하고 나타나는 **1**의 개수

$fsum()$ 의 시간 복잡도는

$$T(n) = O(\log_2 n) + O(\log_2 n) = O(\log_2 n)$$



행렬의 거듭제곱 문제:

$n \times n$ 크기의 행렬 A 와 양의 정수 m 이 주어질 때,

행렬 A 를 연속해서 m 번 곱한 A 의 거듭제곱(power) A^m 을 구하시오.

나이브한 접근법: 행렬 A 를 m 번 곱한다.

행렬 곱셈 알고리즘: $T(n) = O(n^3)$

행렬의 거듭제곱: $T(n) = O(n^3 m)$

행렬 곱셈을 더 빨리 할 수 있다면?

Strassen 행렬 곱셈 알고리즘: $T(n) = O(n^{\log_2 7})$

행렬의 거듭제곱: $T(n) = O(n^{\log_2 7} m)$



행렬의 거듭제곱 문제: 분할정복 알고리즘

$$A^m = A^{m/2} \times A^{m/2}, m \text{은 짝수}$$

$$A^m = A \times A^{m-1}, m \text{은 홀수}$$

$$A^1 = A$$

$$A^0 = I \text{ (항등 행렬)}$$

```
def mpow(A, m):  
    if m == 1:  
        return A  
    elif m == 0:  
        return identity(A)  
    elif m % 2 == 1: # m이 홀수면  
        return mmult(A, mpow(A, m - 1))  
    else: # m이 짝수면  
        half = mpow(A, m // 2)  
        return mmult(half, half)
```



행렬의 거듭제곱 문제: m 이 홀수일 때의 분할과 시간 복잡도

m 이 홀수일 때, $A \times A^{m-1}$ 으로 나누지 않고, 좀 더 절반에 가깝게 나누는 게 좋지 않을까?

예) A^7 을 $A \times A^6$ 으로 나누는 것이 아니라, $A^3 \times A^4$ 으로 나누면?

대부분의 분할 정복 알고리즘은 가능한 한 절반에 가깝게 문제를 나누고자 한다.

예) 퀵 정렬의 *partition()* 알고리즘: *pivot*의 선정 방법

하지만, 이 문제에서 이렇게 나누는 것은

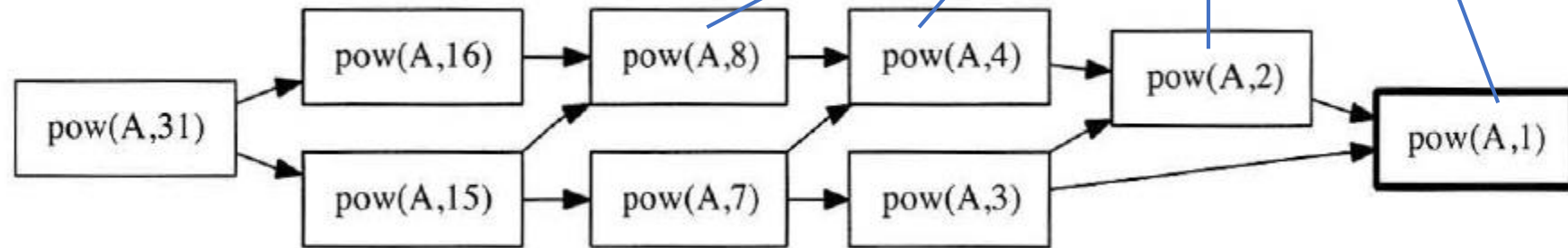
부분 문제가 중복되는 문제(*overlapping subproblem*)가 발생



Lec. 03. 분할 정복

중복으로 계산하는 일이 많이 발생

(a) 홀수를 절반으로 나누는 경우



(b) 홀수에서 1을 빼서 짝수로 만드는 경우

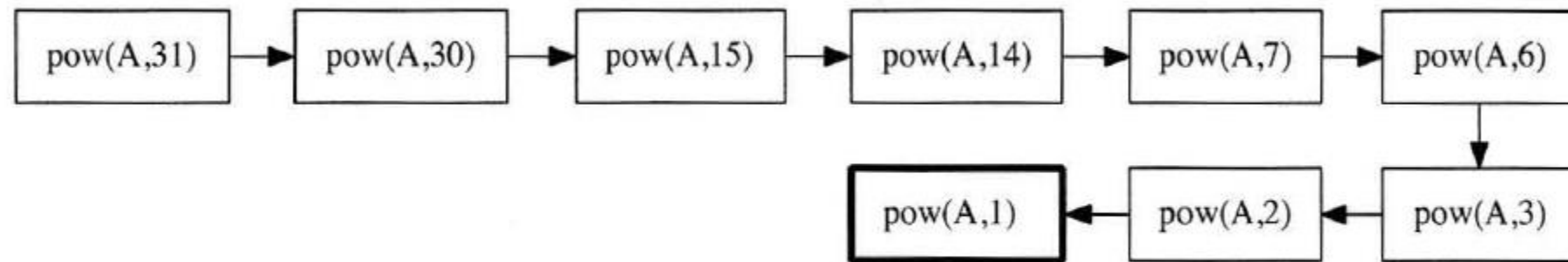


그림 7.2 행렬 거듭제곱을 위한 두 가지 분할 방식의 비교그림



분할정복 알고리즘은 시간 복잡도를 개선할 수 있는가?

예제: 병합 정렬과 퀵 정렬

Merge Sort: 입력을 거의 절반으로 쪼개어 병합을 $O(n)$ 시간에 수행 $O(n \cdot \lg n)$

Quick Sort: 입력을 되도록 절반으로 분할. 최악은 $O(n^2)$, 평균적으로 $O(n \cdot \lg n)$

예제: 카라츠바 알고리즘과 슈트라쎈 알고리즘

입력을 n 개로 나누되, 재귀 호출을 n 보다 작게 할 수 있다면?

Karatsuba 알고리즘: 큰 정수의 곱셈 $O(n^{\log_2 3}) \approx O(n^{1.585})$

Strassen 알고리즘: 큰 행렬의 곱셈 $O(n^{\log_2 7}) \approx O(n^{2.81})$



마스터 정리: *Master Theorem*

분할정복 알고리즘의 시간 복잡도를 분석하는데 유용한 수학적 정리

$$T(n) = a \times T\left(\frac{n}{b}\right) + \Theta(n^k), \quad a > 0, b > 1$$

- $a > b^k: T(n) \in \Theta(n^{\log_b a})$
- $a = b^k: T(n) \in \Theta(n^k \log n)$
- $a < b^k: T(n) \in \Theta(n^k)$



다음과 같은 시간 복잡도를 가진 알고리즘의 성능을 빅오 표기법으로 표현하시오.

- $T(n) = 8 \times T\left(\frac{n}{2}\right) + 1000n^2$
- $T(n) = 4 \times T\left(\frac{n}{4}\right) + 10n$
- $T(n) = 2 \times T\left(\frac{n}{2}\right) + 100n^2$

카라츠바 알고리즘:

$$T(n) = 3 \times T\left(\frac{n}{2}\right) + O(n)$$

쉬트라센 알고리즘:

$$T(n) = 7 \times T\left(\frac{n}{2}\right) + O(n^2)$$



Lec. 03. 분할 정복

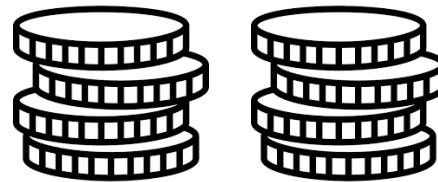
분할 정복을 할 때, **몇 개로 나누는 것이 적당할까?**

가짜 동전 문제: *Fake Coin Problem*

n 개의 동전이 주어졌는데, 이 중 **하나**는 다른 동전보다 더 **가벼운 가짜 동전**이다.

균형 저울을 사용하여 이 중에서 가짜 동전이 어떤 동전인지 찾고자 한다.

저울의 사용 횟수를 최소로 하고 싶을 때, **최소 몇 번** 만에 동전을 찾을 수 있을까?



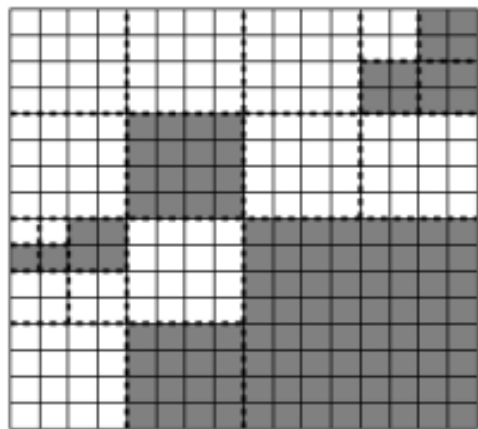
$$n = 8$$



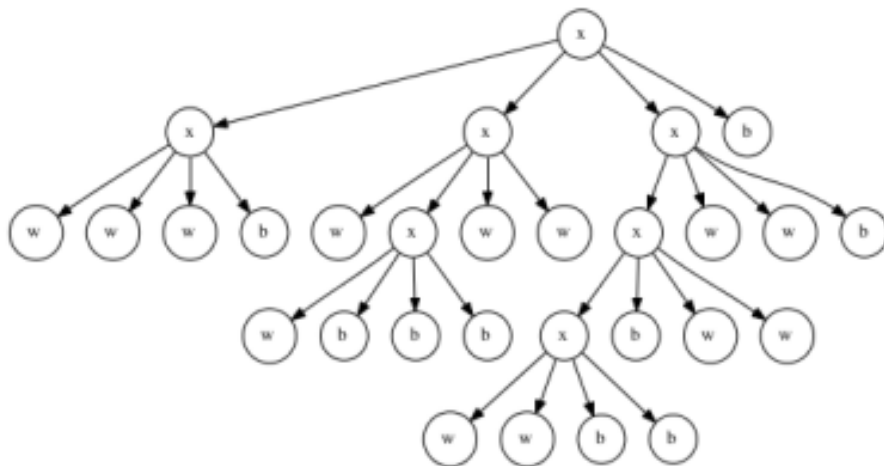
AL2.3.1: 쿼드 트리 뒤집기 (ID: QUADTREE)

문제

대량의 좌표 데이터를 메모리 안에 압축해 저장하기 위해 사용하는 여러 기법 중 쿼드 트리(*quad tree*)란 것이 있습니다. 주어진 공간을 항상 4개로 분할해 재귀적으로 표현하기 때문에 쿼드 트리라는 이름이 붙었는데, 이의 유명한 사용처 중 하나는 검은 색과 흰 색밖에 없는 흑백 그림을 압축해 표현하는 것입니다. 쿼드 트리는 $2N \times 2N$ 크기의 흑백 그림을 다음과 같은 과정을 거쳐 문자열로 압축합니다.



(a)



(b)



Lec. 03. 분할 정복

- 이 그림의 **모든 픽셀이 검은 색일 경우** 이 그림의 쿼드 트리 압축 결과는 그림의 크기에 관계없이 ***b***가 됩니다.
- 이 그림의 **모든 픽셀이 흰 색일 경우** 이 그림의 쿼드 트리 압축 결과는 그림의 크기에 관계없이 ***w***가 됩니다.
- 모든 픽셀이 같은 색이 아니라면**, 쿼드 트리는 이 그림을 가로 세로로 각각 2등분해 **4개의 조각으로 쪼갠 뒤 각각을 쿼드 트리 압축**합니다. 이때 전체 그림의 압축 결과는 ***x***(왼쪽 위 부분의 압축 결과)(오른쪽 위 부분의 압축 결과)(왼쪽 아래 부분의 압축 결과)(오른쪽 아래 부분의 압축 결과)가 됩니다. 예를 들어 그림 (a)의 왼쪽 위 4분면은 **xwwwb**로 압축됩니다.

그림 (a)와 그림 (b)는 16×16 크기의 예제 그림을 쿼드 트리가 어떻게 분할해 압축하는지를 보여줍니다. 이때 전체 그림의 압축 결과는 **xxwww bxwxw bbbww xxxww bbbww wwbb**가 됩니다.

쿼드 트리로 압축된 흑백 그림이 주어졌을 때, 이 그림을 **상하로 뒤집은 그림**을 **쿼드 트리 압축**해서 출력하는 프로그램을 작성하세요.



Lec. 03. 분할 정복

입력

첫 줄에 테스트 케이스의 개수 C ($C \leq 50$)가 주어집니다. 그 후 C 줄에 하나씩 쿼드 트리로 압축한 그림이 주어집니다. 모든 문자열의 길이는 1,000 이하이며, 원본 그림의 크기는 $2^{20} \times 2^{20}$ 을 넘지 않습니다.

출력

각 테스트 케이스당 한 줄에 주어진 그림을 상하로 뒤집은 결과를 쿼드 트리 압축해서 출력합니다.



Lec. 03. 분할 정복

예제 입력

4
w
xbwwb
xbwxwbbwb
xxwwwbxwxwbbbwxxwbbbwbb

예제 출력

w
xwbbw
xxbwwbbbw
xxwbxwwxbbwbbwbxwbwwwxwwwxbbwb



무식하게 풀 수 있을까?

주어진 쿼드트리로 압축한 문자열을 풀어서 **실제 이미지를 얻고**, (*decompress*)

이 이미지를 상하 반전한 뒤 **다시 쿼드트리로 압축하여 출력** (*compress*)

이미지의 크기가 최대 $2^{20} \times 2^{20}$ 이므로 **시간/공간 효율성에 문제가 있음**

압축을 해제한 결과를 저장하지 않고, **뒤집은 결과를 곧장 생성해야 함**

일단, **압축 알고리즘**과 **해제 알고리즘**을 구현한 뒤,

압축을 해제하지 않고 바로 해결하는 뒤집기 알고리즘을 구현해보자.



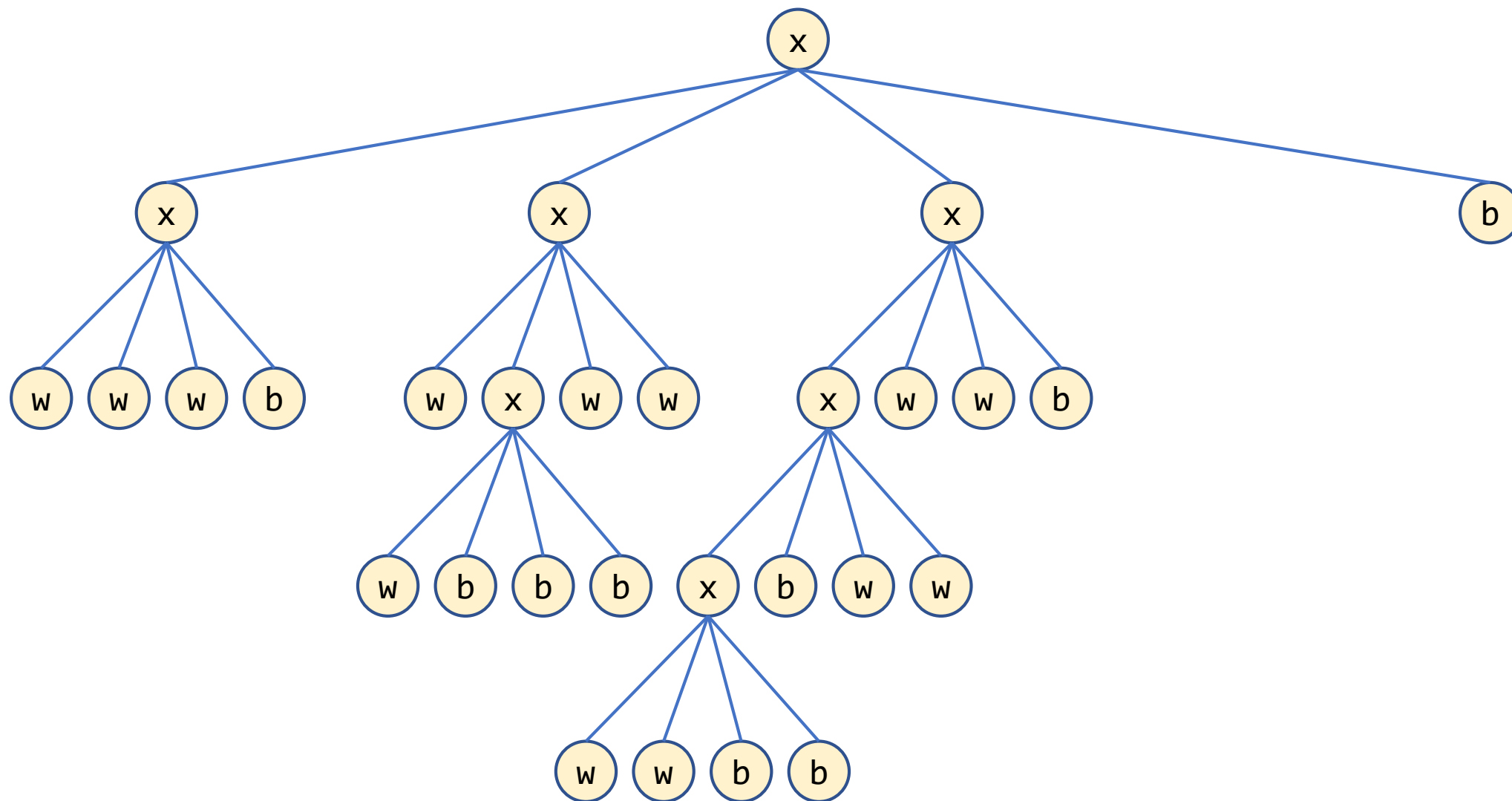
쿼드 트리의 압축 풀기: *decompress*

종료 조건: 주어진 문자열 *quadtrees*의 첫 글자가 w 이거나 b 인 경우
해당하는 크기의 모든 픽셀을 해당 색으로 채우고 종료

재귀 조건: 주어진 문자열 *quadtrees*의 첫 글자가 x 인 경우
문자열의 나머지 부분을 **넷으로 쪼개어** 재귀 호출



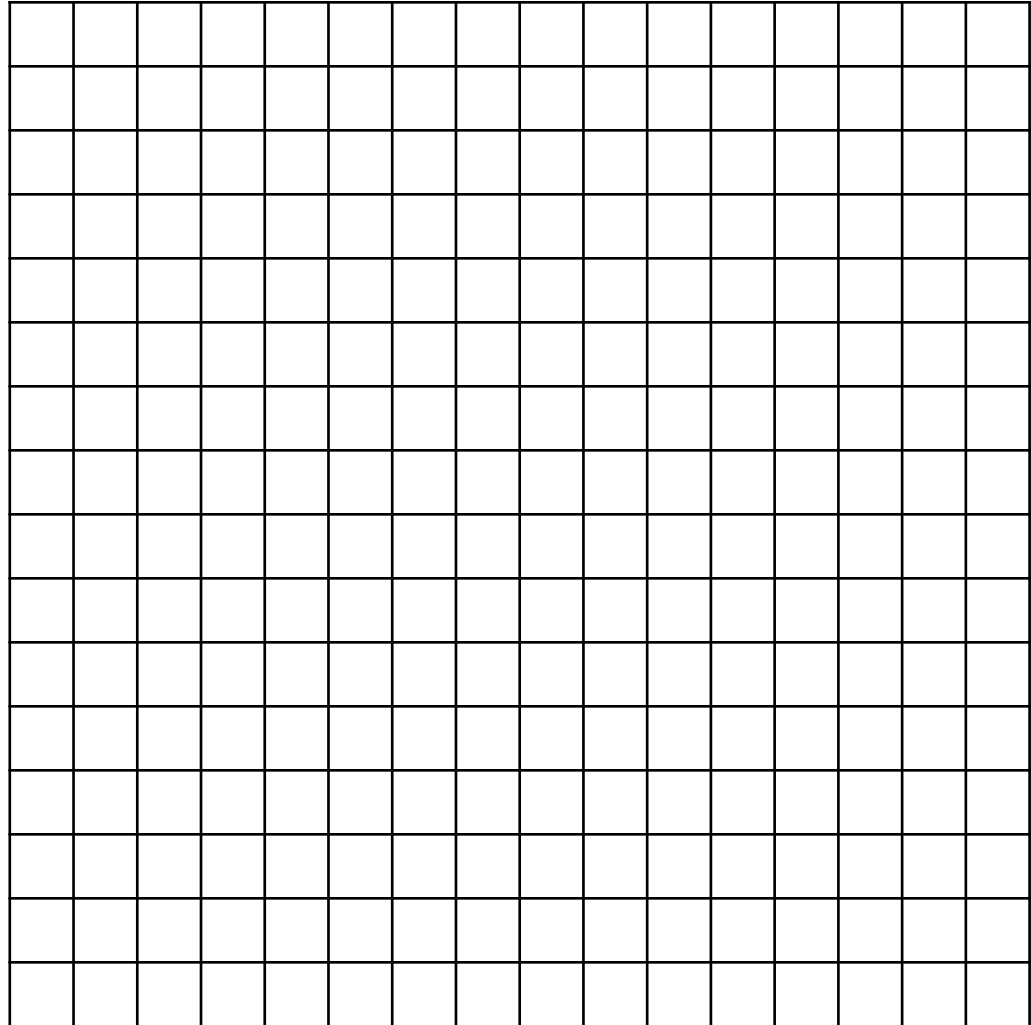
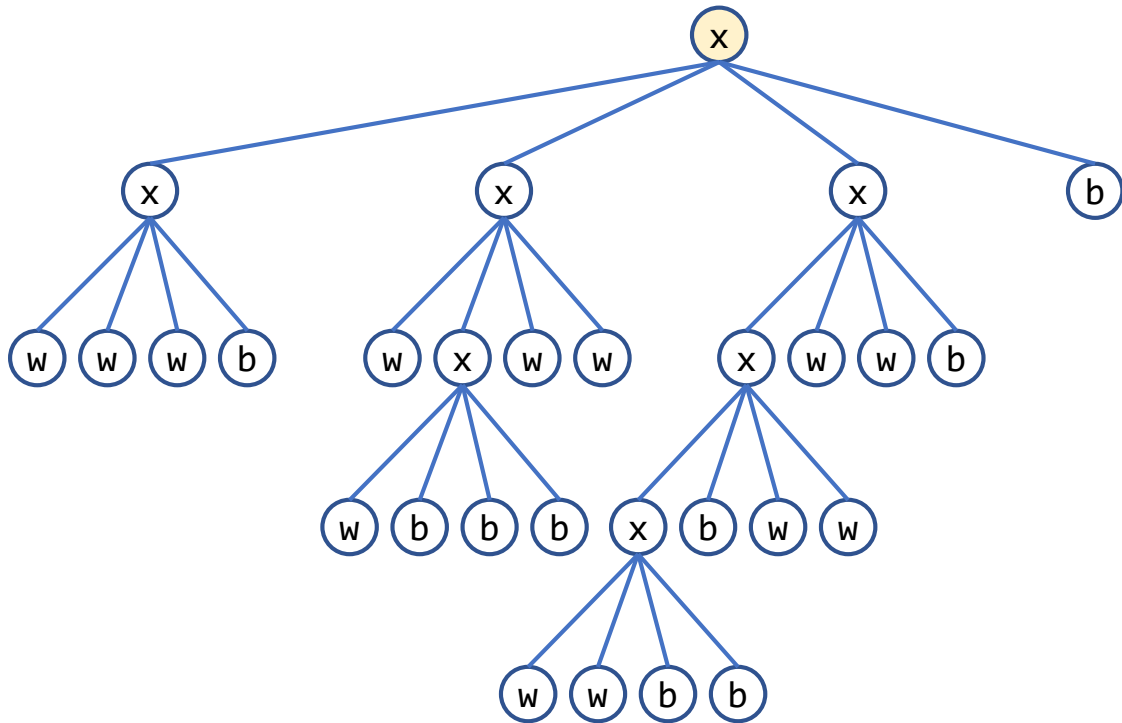
quadtree = *xxwwwbxxwbbbwxxxwbbbwwwwbb*





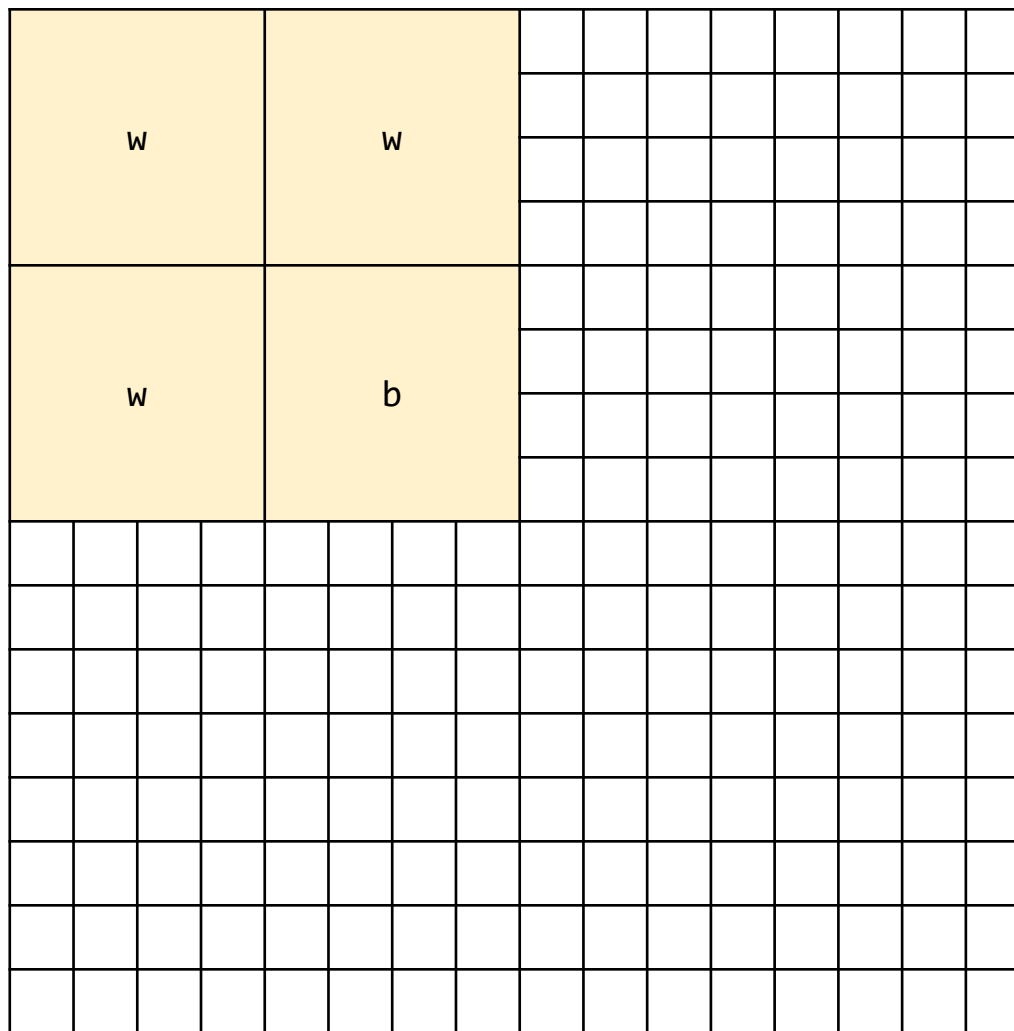
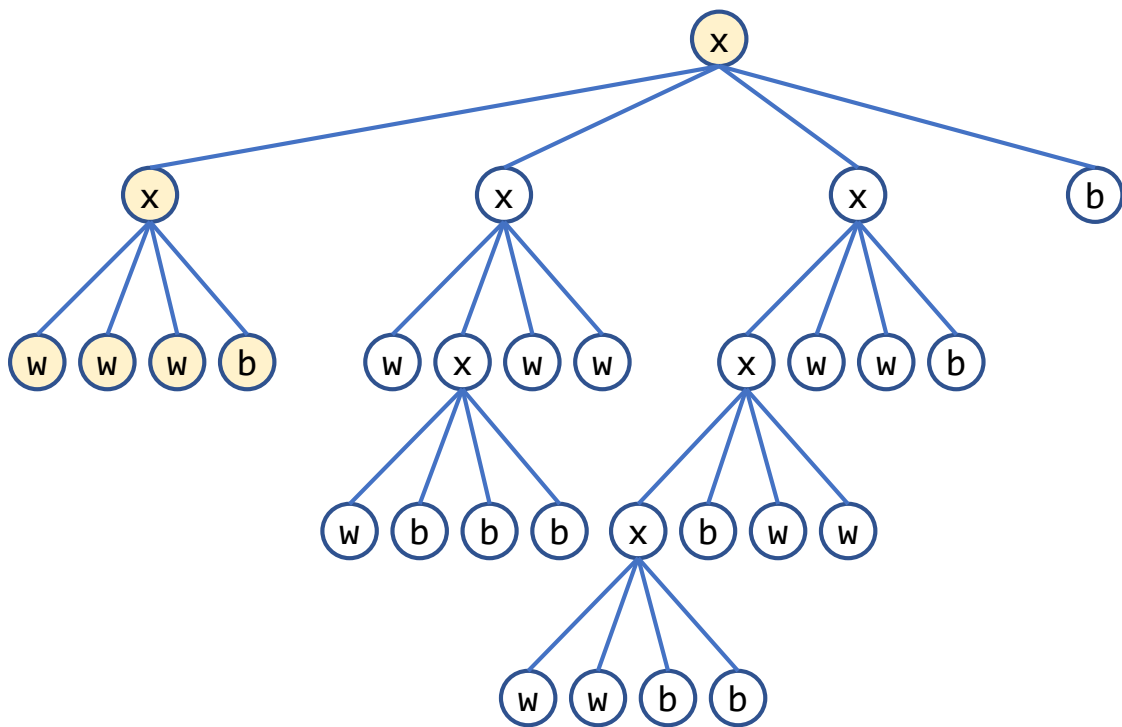
Lec. 03. 분할 정복

quadtree = *xxwwwbxxwbbbwxxxwwbbbwwwwbb*





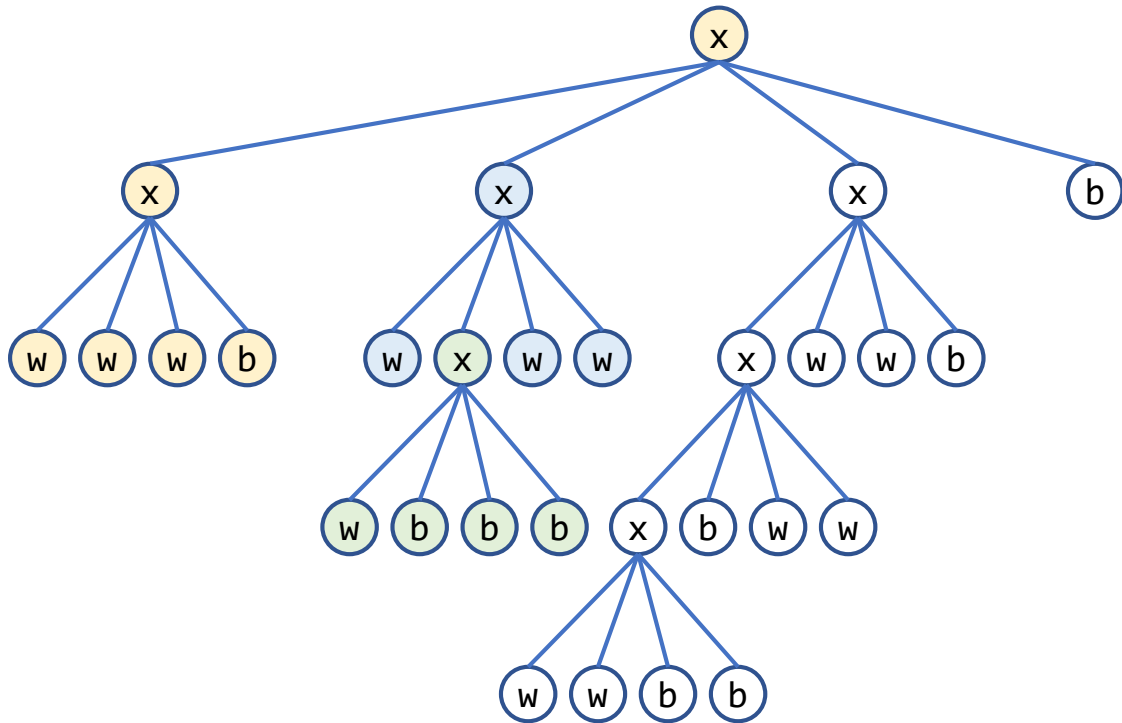
quadtree = *xxwwwbxxwbbbwwwxxxwwbbbwwwbb*





Lec. 03. 분할 정복

quadtree = *xxwwwbxxwbbbwxxxwwbbbwwwwbb*

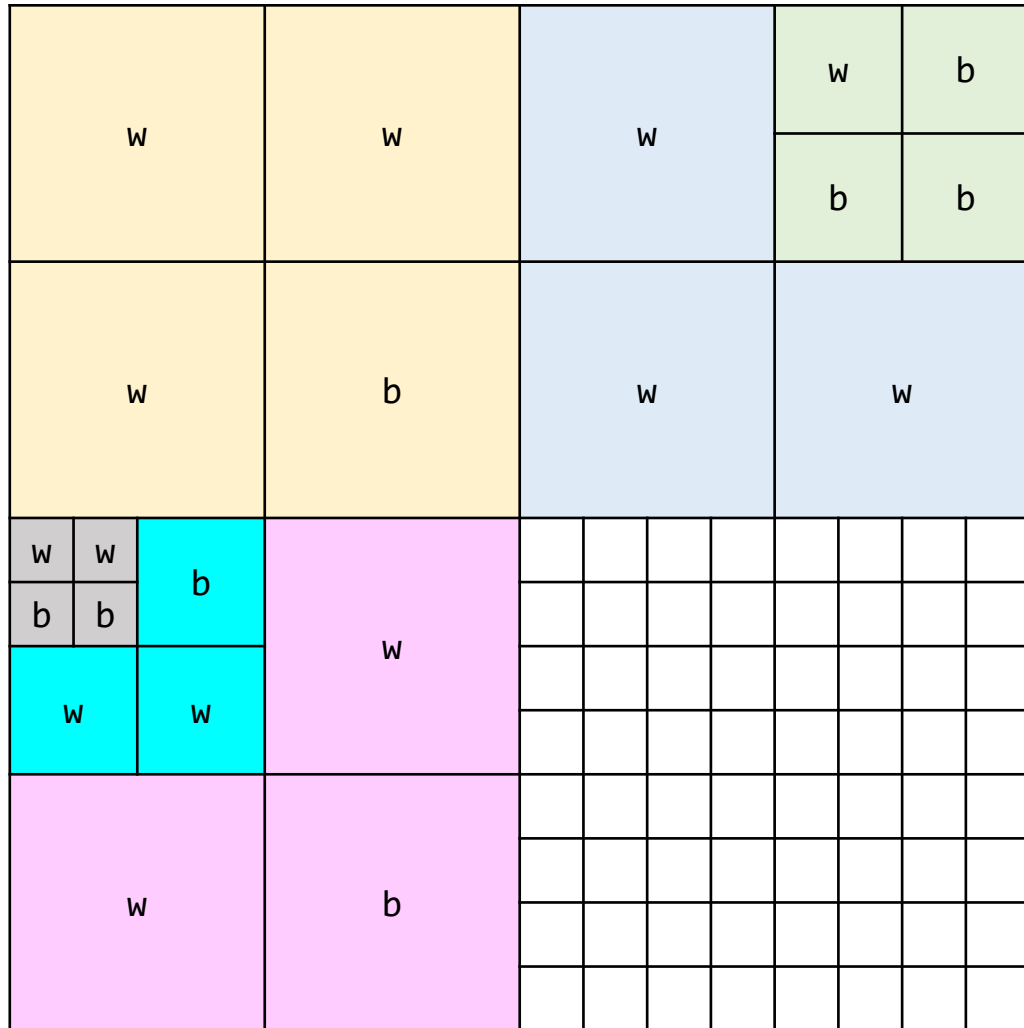
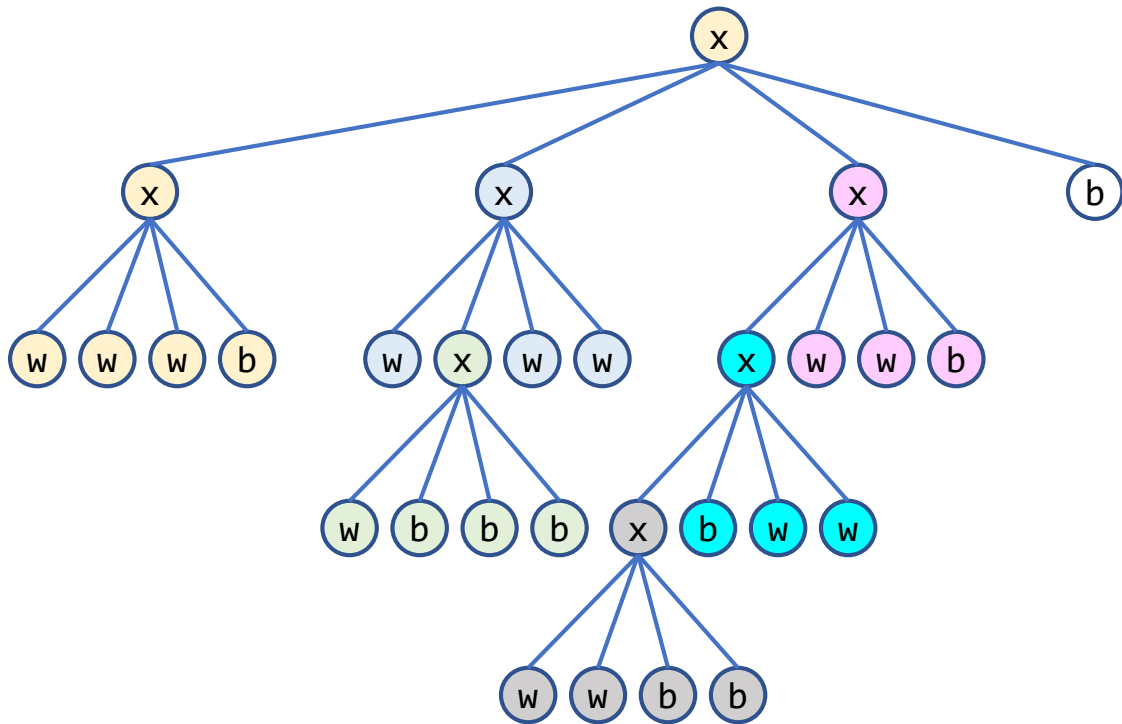


w				w				w				w		b	
w				b				w				b		b	



Lec. 03. 분할 정복

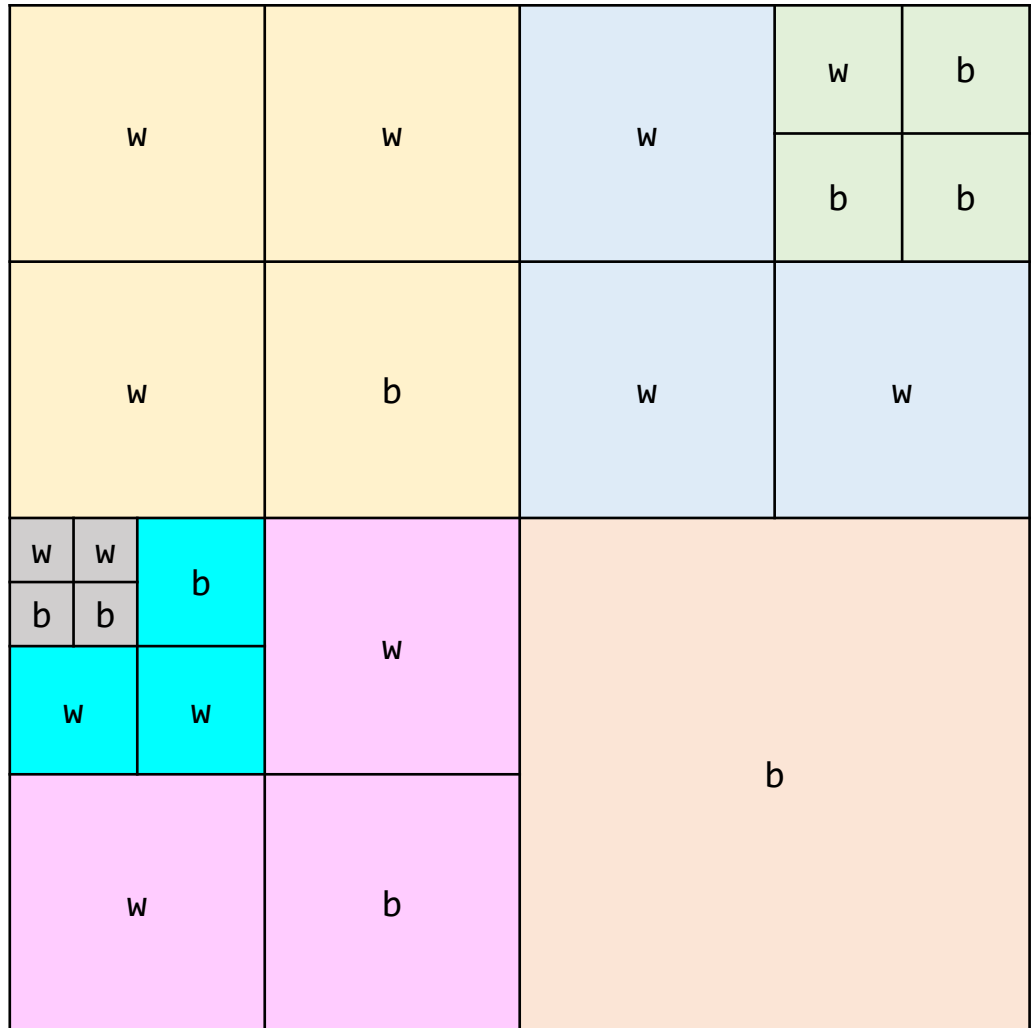
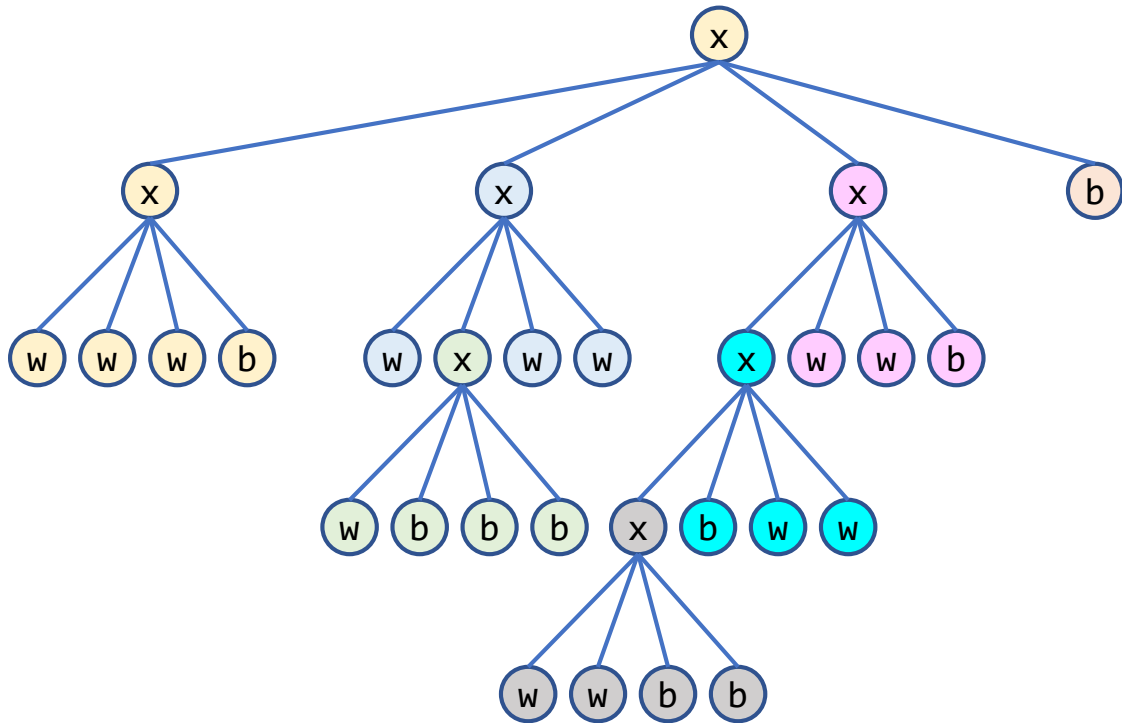
quadtree = *xxwwwbxxwbbbwwwxxxwwbbbwwwbb*





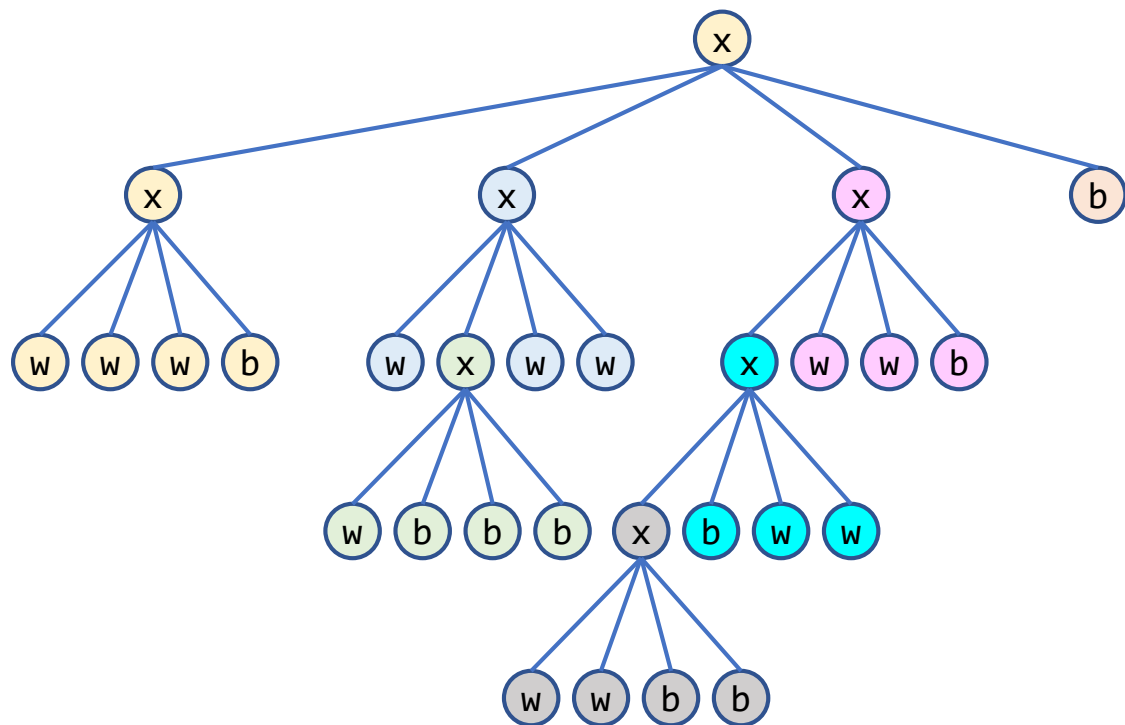
Lec. 03. 분할 정복

quadtree = *xxwwwbxxwbbbwxxxwwbbbwwwwbb*





quadtree = *xxwwwbxxwbbbwwwxxxwwbbbwwwbb*



0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0	1	1	1	1	1	1	1	1
1	1	1	1	0	0	0	0	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1



```
'''
```

decompress.py: 쿼드 트리 압축 해제

```
'''
```

```
quadtree = "xxwwwbxxwbbbwxxxwwbbbwbbwbb"
```

```
size = 16 # 압축 해제 문제는 크기가 주어져야 함
```

```
decompressed = [[0] * size for _ in range(size)]
```

```
get_next.idx = 0 # static 변수로 인덱스를 초기화
```

```
decompress(quadtree, 0, 0, 16, decompressed)
```

```
for i in range(len(decompressed)):
```

```
    print(*decompressed[i])
```



주어진 쿼드 트리 문자열을 어떻게 넷으로 쪼개지?

x 다음에 오는 문자열의 각 부분의 길이가 일정하지 않으므로, 넷으로 쪼개기가 어려움
문자열의 인덱스를 하나씩 이동시키면서 필요한 만큼 가져다 쓰는 방법을 이용

static 변수를 이용하여 한 글자를 검사할 때마다 한 칸씩 앞으로 이동

```
def get_next(quadtrees):  
    get_next.idx += 1  
    return quadtrees[get_next.idx - 1]
```



Lec. 03. 분할 정복

```
# quadtree: 쿼드 트리 압축 문자열
# y, x, size: 압축을 해제할 시작 좌표(y, x)와 이미지 크기(size * size)
# decompressed[i][j]: 픽셀 값. 0이면 흰 색, 1이면 검은 색
def decompress(quadtree, y, x, size, decompressed):
    head = get_next(quadtree) # 문자열에서 다음 문자를 가져온다.
    if head in "bw": # 흰 색이거나 검은 색이면
        for i in range(y, y + size):
            # 해당하는 크기의 모든 픽셀을 해당하는 색으로 채운다.
            decompressed[i][x:x+size] = [(head=="b")*1] * size
    else: # 재귀 호출
        half = size // 2
        decompress(quadtree, y, x, half, decompressed)
        decompress(quadtree, y, x + half, half, decompressed)
        decompress(quadtree, y + half, x, half, decompressed)
        decompress(quadtree, y + half, x + half, half, decompressed)
```




쿼드 트리 압축하기: *compress*

종료 조건: 주어진 범위 내의 모든 픽셀이 흰 색이거나 검은 색인 경우
흰 색이면 w 를, 검은 색이면 b 를 반환하고 종료

재귀 조건: 주어진 범위 내에 흰 색과 검은 색이 섞여 있는 경우
주어진 범위를 절반의 크기로 네 부분으로 자르고,
각 조각들을 재귀 호출하여 네 개의 압축 문자열 각각 구한 뒤,
문자열 x 와 네 조각의 압축 문자열을 결합하여 반환



Lec. 03. 분할 정복

```
'''
```

compress.py: 쿼드 트리 압축하기

```
'''
```

```
quadtree = [
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
    [0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
    [1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
    [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
    [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
    [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]]

print(compress(quadtree, 0, 0, len(quadtree)))
```



Lec. 03. 분할 정복

```
# 주어진 시작 좌표(y, x)와 크기(size) 범위에서
# 모두 흰 색(합이 0)이거나 모두 검은 색(합이 size * size)인가를 확인
def check(quadtree, y, x, size):
    s = sum([sum(quadtree[i][x:x+size]) for i in range(y, y + size)])
    return "w" if s == 0 else "b" if s == size * size else "x"
```

범위 내의 픽셀 값을 모두 합하면
0이 됨

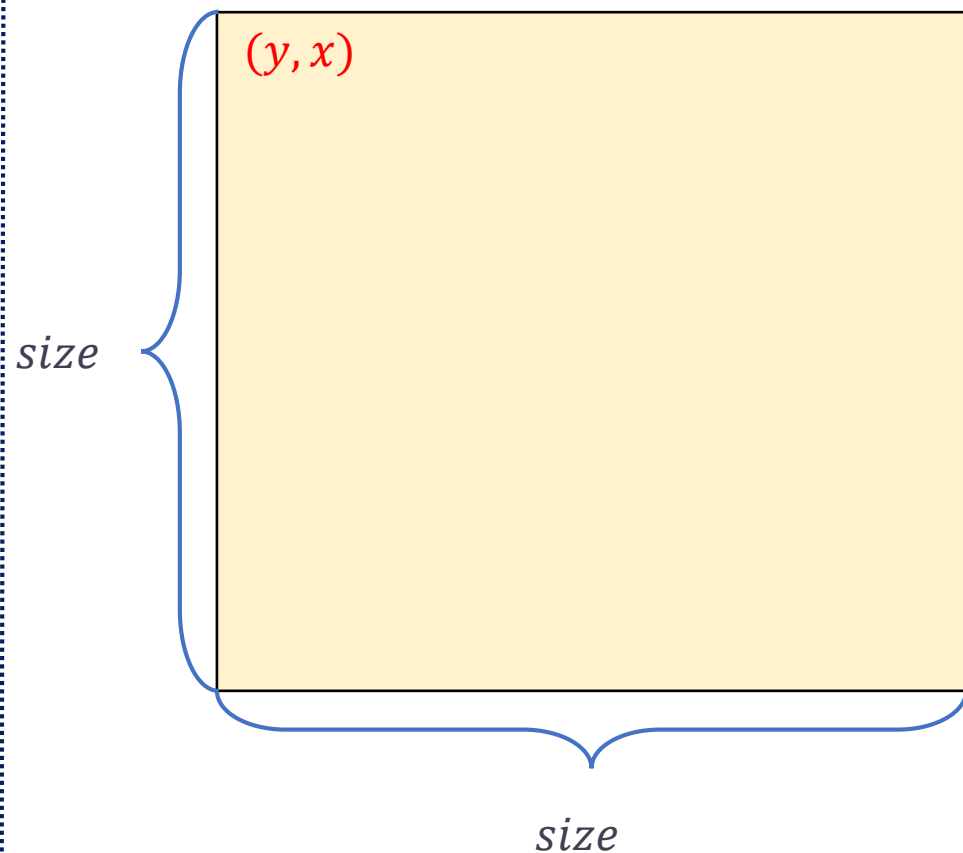
0	0	1	0
0	0	1	1
1	1	1	1
0	1	1	1

0과 1이 섞여 있으면
 $0 < s < size \times size$

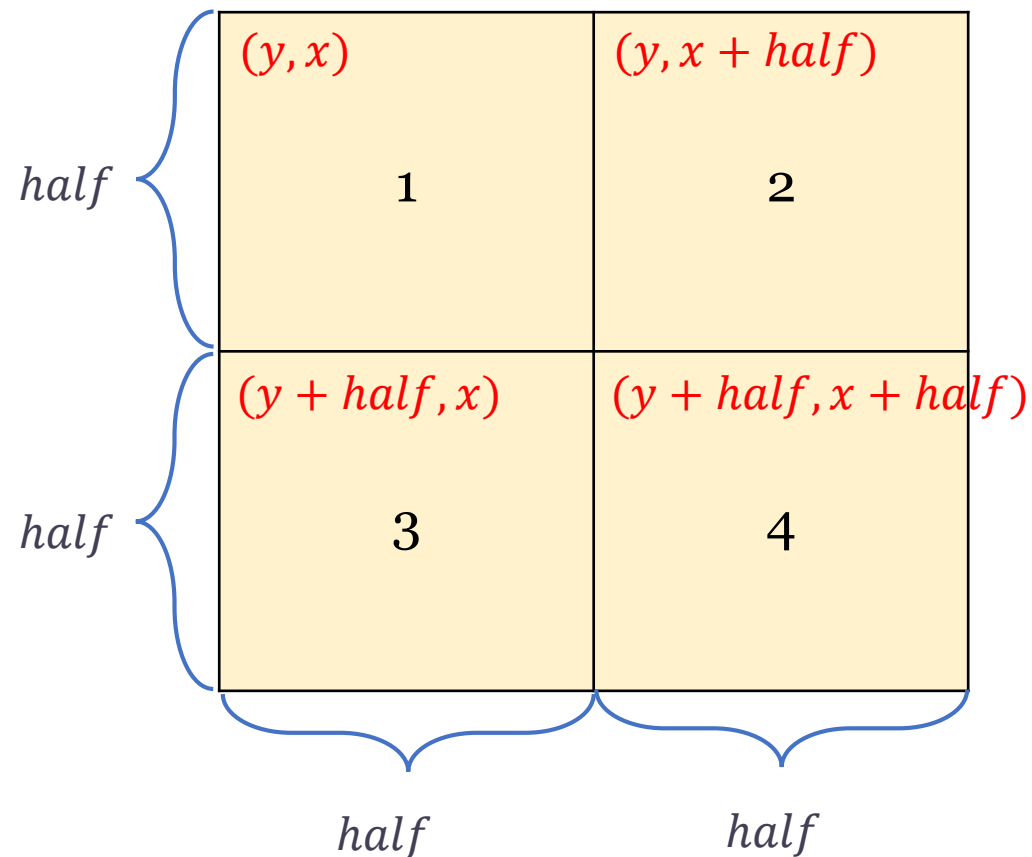
범위 내의 픽셀 값을 모두 합하면
 $size \times size$ 가 됨



쿼드 트리를 네 부분으로 분할하기



$$half = size/2$$





```
# quadtree[i][j]: 픽셀 데이터. 0이면 흰 색, 1이면 검은 색
# y, x, size: 압축을 시작하는 좌표(y, x)와 이미지 크기(size)
def compress(quadtree, y, x, size):
    ret = check(quadtree, y, x, size)
    if ret != "x": # 모두 흰 색이거나 검은 색이면
        return ret # 해당 색을 반환하고 종료
    else:
        half = size // 2 # 크기를 절반으로 자른다.
        qt1 = compress(quadtree, y, x, half) # 1: 상단 왼쪽 조각
        qt2 = compress(quadtree, y, x + half, half) # 2: 상단 오른쪽 조각
        qt3 = compress(quadtree, y + half, x, half) # 3: 하단 왼쪽 조각
        qt4 = compress(quadtree, y + half, x + half, half) # 4: 하단 오른쪽 조각
        return "x" + qt1 + qt2 + qt3 + qt4
```



쿼드 트리 뒤집기 문제로 돌아와서:

압축을 풀지 않고도 쿼드 트리를 뒤집으려면?

종료 조건: 전체가 한 가지 색일 경우 뒤집을 필요가 없다.

주어진 문자열(w 또는 b)를 그대로 반환하고 종료

재귀 조건: 두 가지 색이 섞여 있을 경우

네 부분으로 쪼개어 각각을 상하로 뒤집은 다음(재귀 호출),

이들을 상하로 뒤집히도록 병합하여 반환한다.

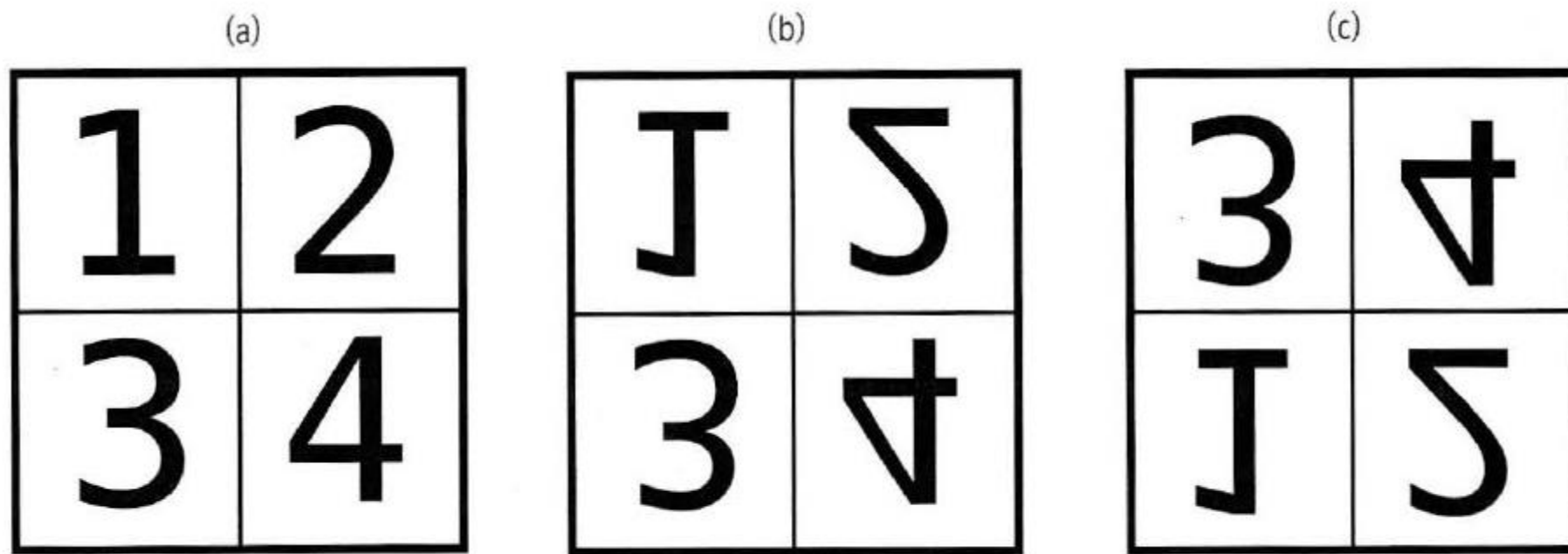


그림 7.6 N개의 원소를 절반씩 나눠 뒤집기



```
...
```

P.3.1. 쿼드 트리 뒤집기 (ID: QUADTREE)

```
...
```

```
for i in range(int(input())):
```

```
    quadtree = input()
```

```
    get_next.idx = 0 # 정적 변수로 인덱스를 초기화
```

```
    reversed = quadtree_reverse(quadtree)
```

```
    print(reversed)
```

```
# 정적 변수를 이용하여 quadtree에서 한 문자씩 꺼내어 반환
```

```
def get_next(quadtree):
```

```
    get_next.idx += 1 # 인덱스를 다음 위치로 이동한 후
```

```
    return quadtree[get_next.idx - 1] # 이전 인덱스 값을 리턴
```




```
# 쿼드 트리를 뒤집어서 반환
def quadtree_reverse(quadtree):
    head = get_next(quadtree)
    if head in "bw": # 한 가지 색일 경우
        return head # 해당하는 색을 반환
    else:
        # 네 부분으로 4분할하여 각각을 뒤집는다.
        qt1 = quadtree_reverse(quadtree)
        qt2 = quadtree_reverse(quadtree)
        qt3 = quadtree_reverse(quadtree)
        qt4 = quadtree_reverse(quadtree)
        # 4분할한 조각들의 위치를 상하로 뒤집는다.
        return "x" + qt3 + qt4 + qt1 + qt2
```



쿼드 트리 뒤집기 알고리즘의 시간 복잡도 분석:

quadtree_reverse() 함수는 한 번 호출될 때마다
주어진 문자열에서 한 문자씩 앞으로 이동한다.

따라서, 함수의 호출 횟수가

문자열의 길이(n)에 선형적으로 비례하므로 시간 복잡도는 $T(n) = O(n)$

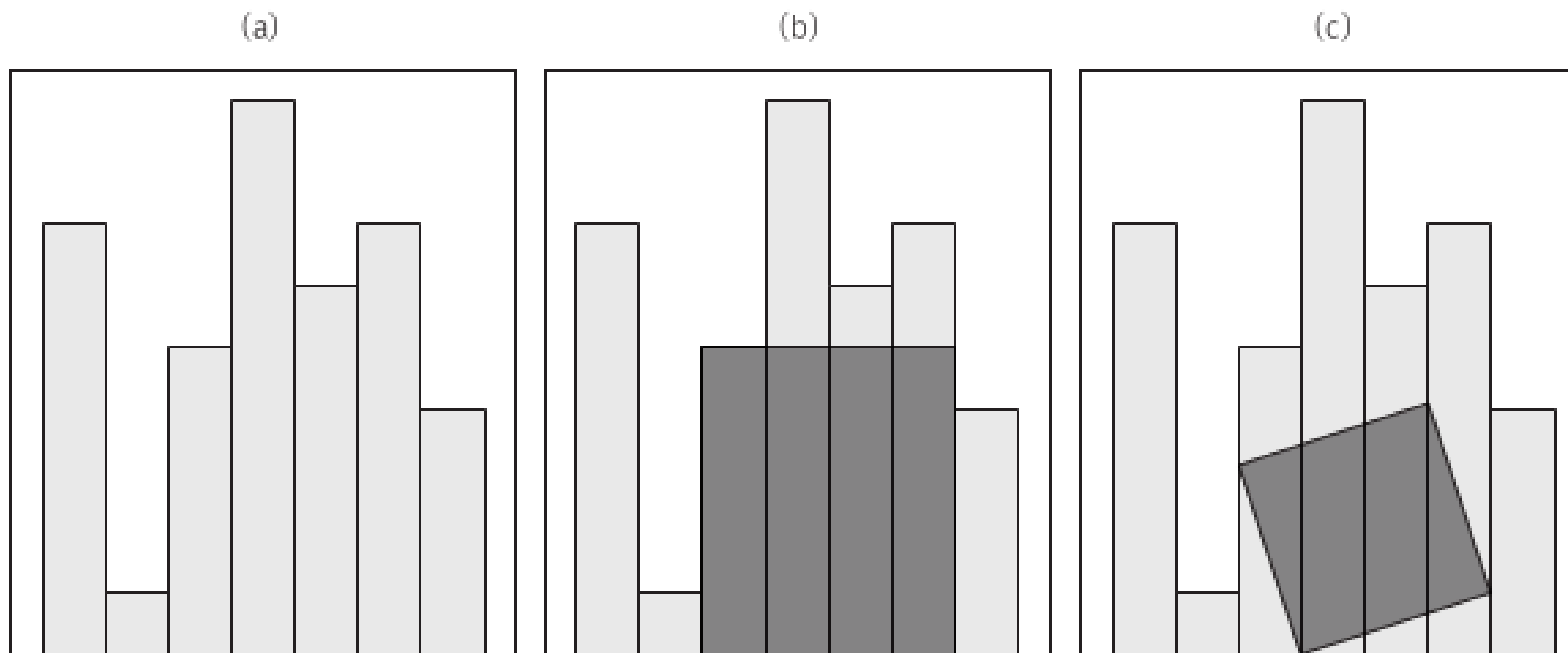


AL2.3.2: 울타리 잘라내기 (ID: FENCE)

문제

너비가 같은 N 개의 나무 판자를 붙여 세운 울타리가 있습니다. 시간이 지남에 따라 판자들이 부러지거나 망가져 높이가 다 달라진 관계로 울타리를 통째로 교체하기로 했습니다. 이때 버리는 울타리의 일부를 직사각형으로 잘라내 재활용하고 싶습니다. 그림 (b)는 (a)의 울타리에서 잘라낼 수 있는 많은 직사각형 중 가장 넓은 직사각형을 보여줍니다. 울타리를 구성하는 각 판자의 높이가 주어질 때, 잘라낼 수 있는 직사각형의 최대 크기를 계산하는 프로그램을 작성하세요. 단 (c)처럼 직사각형을 비스듬히 잘라낼 수는 없습니다.

판자의 너비는 모두 1이라고 가정합니다.





Lec. 03. 분할 정복

입력

첫 줄에 테스트 케이스의 개수 C ($C \leq 50$)가 주어집니다. 각 테스트 케이스의 첫 줄에는 판자의 수 N ($1 \leq N \leq 20000$)이 주어집니다. 그 다음 줄에는 N 개의 정수로 왼쪽부터 각 판자의 높이가 순서대로 주어집니다. 높이는 모두 10,000 이하의 음이 아닌 정수입니다.

출력

각 테스트 케이스당 정수 하나를 한 줄에 출력합니다. 이 정수는 주어진 울타리에서 잘라낼 수 있는 최대 직사각형의 크기를 나타내야 합니다.





Lec. 03. 분할 정복

예제 입력

```
3
7
7 1 5 9 6 7 3
7
1 4 4 4 4 1 1
4
1 8 2 2
```

예제 출력

```
20
16
8
```



'''

P.3.2: 울타리 잘라내기 (ID: FENCE): 분할정복 알고리즘

'''

```
for _ in range(int(input())):  
    n = int(input())  
    h = list(map(int, input().split()))  
    print(fence(h, 0, n - 1))
```



무식하게 풀 수 있을까?

각 판자 높이의 배열 $h[]$ 가 주어질 때, l 번 판자부터 r 번 판자까지 잘라낸다고 하자.

$$\text{사각형의 넓이} = (r - l + 1) \times \min_{i=l..r} h[i]$$

2중 for문으로 가능한 모든 l 과 r 을 순회하며 위 값을 계산하는 방법:

슬라이딩 윈도우를 이용해도 시간 복잡도가 $O(n^2)$ 이므로

최대 입력 크기 20,000에 대해서는 시간 초과를 피하기 어렵다.



분할해서 정복할 수 있을까?

n 개의 판자를 절반으로 나눠 두 개의 부분 문제를 만들면?

우리가 찾는 최대 직사각형은 다음 세 가지 중 하나에 속한다.

1. 가장 큰 직사각형을 **왼쪽** 부분 문제에서만 잘라낼 수 있다.
2. 가장 큰 직사각형을 **오른쪽** 부분 문제에서만 잘라낼 수 있다.
3. 가장 큰 직사각형은 **왼쪽 부분 문제와 오른쪽 부분 문제에 걸쳐 있다.**

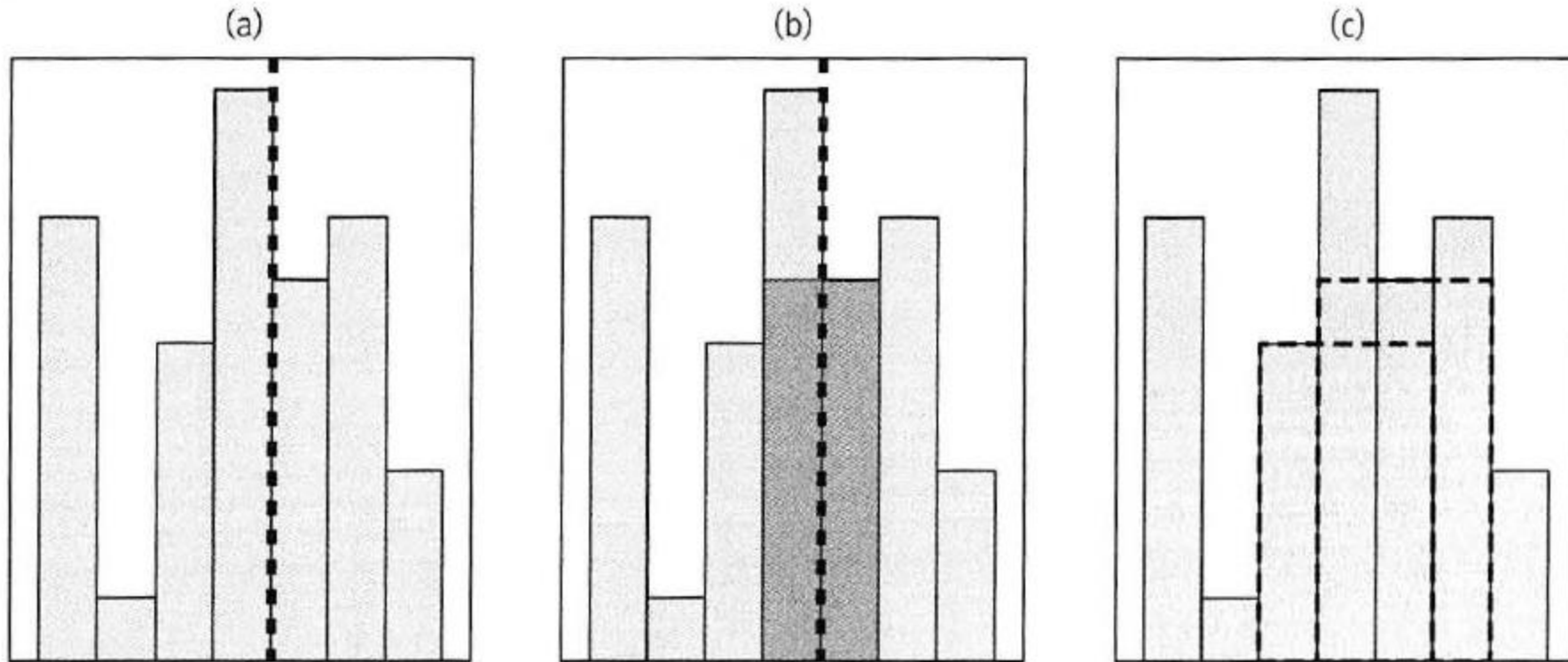


그림 7.7 울타리 잘라내기 문제를 위한 분할 정복 알고리즘



양쪽 부분 문제에 걸친 경우에 가장 큰 직사각형 찾기

힌트: 이 직사각형은 반드시 부분 문제 경계에 있는 두 판자를 포함한다. (그림 7.7(b) 참조)

이 직사각형에서 시작해서 사각형을 가로로 한 칸씩 확대해 나간다고 하자.

왼쪽 끝을 한 칸 왼쪽으로 옮길 수도 있고, 오른쪽 끝을 한 칸 오른쪽으로 옮길 수도 있다.

그림 7.7(c)에 보이는 두 가지 사각형 중 어느 쪽을 선택해야 할까?

전략: 더 높은 쪽의 판자를 포함하게끔 확장하자.

가장 큰 직사각형을 찾으려면 항상 사각형의 높이를 최대화하는 방향으로 확장해야 한다.



Lec. 03. 분할 정복

$$h[lo - 1] < h[hi + 1]$$

$$h[lo - 1] > h[hi + 1]$$

height

left

right

right = hi

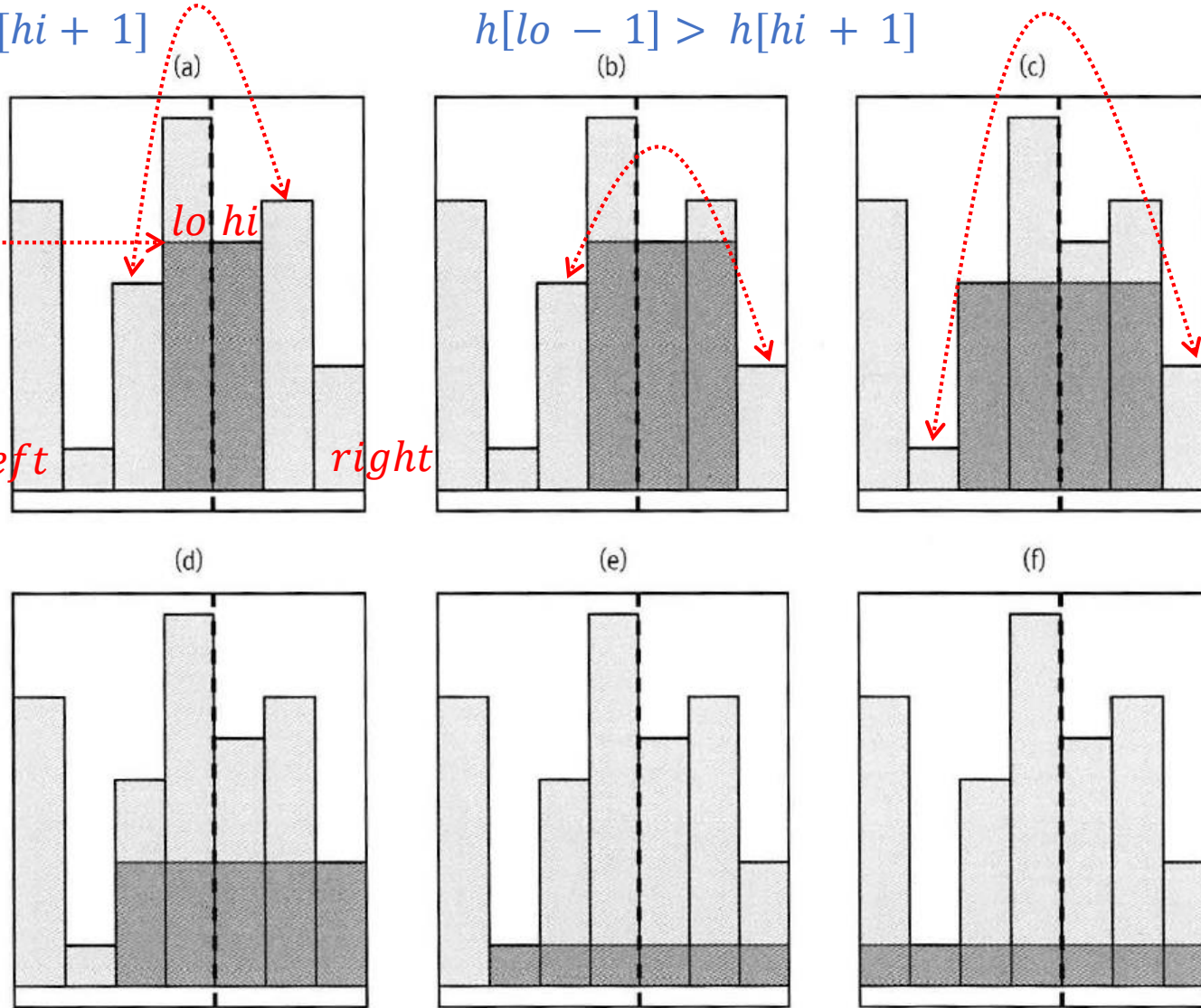


그림 7.8 울타리 잘라내기 문제 병합 알고리즘의 동작 과정



h[left..right] 구간에서 찾아낼 수 있는 가장 큰 직사각형의 넓이 반환

```
def fence(h, left, right):  
    if left == right:  
        return h[left]  
    else:  
        mid = (left + right) // 2  
        # 왼쪽[left,mid]과 오른쪽[mid+1,right] 부분 문제를 각개 격파  
        ret = max(fence(h, left, mid), fence(h, mid + 1, right))  
        # 두 부분에 모두 걸치는 경우의 가장 큰 사각형 고려  
        return max(ret, largest_in_between(h, left, mid, right))
```



두 부분에 모두 걸치는 사각형 중 가장 큰 것을 찾는다.

```
def largest_in_between(h, left, mid, right):
```

```
    low, high = mid, mid + 1 # 경계에 있는 두 판자
```

```
    height = min(h[low], h[high]) # 중에서 더 작은 것을 선택
```

```
    ret = height * 2 # 두 판자에서 잘라낸 사각형의 넓이
```

```
    while left < low or right > high: # 사각형이 입력 전체를 덮을 때까지
```

```
        if high < right and (low == left or h[low - 1] < h[high + 1]):
```

```
            high += 1 # 오른쪽으로 한 칸 확장
```

```
            height = min(height, h[high])
```

```
        else:
```

```
            low -= 1 # 왼쪽으로 한 칸 확장
```

```
            height = min(height, h[low])
```

```
    ret = max(ret, height * (high - low + 1)) # 확장 후 사각형의 넓이 업데이트
```

```
    return ret
```



울타리 잘라내기 분할정복 알고리즘의 시간 복잡도 분석:

이 알고리즘은 n 개의 크기를 가진 입력을 (대략) $\frac{n}{2}$ 개로 나누어 각자 해결

두 부분에 걸쳐 있는 사각형을 찾을 때는 너비가 n 인 사각형을 하나하나 검사하므로 $O(n)$

시간 복잡도: $T(n) = 2 \times T\left(\frac{n}{2}\right) + O(n)$

마스터 정리에 의해 $T(n) = O(n \log n)$



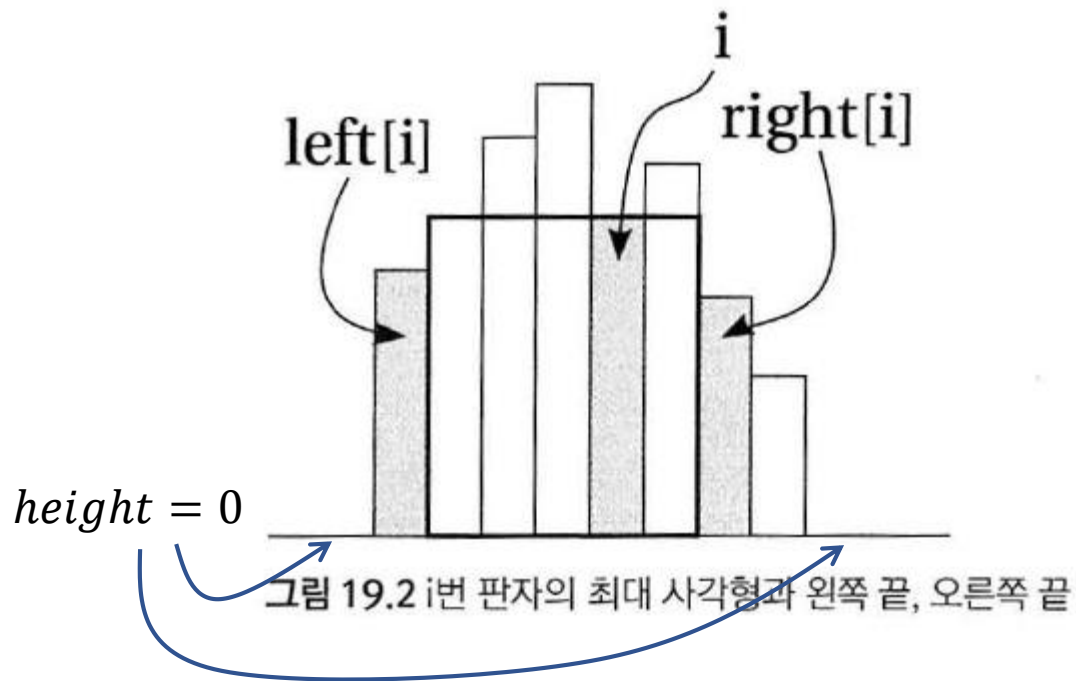
울타리 잘라내기 문제를 다른 방법으로도 풀 수 있을까?

- 스위핑(15.11절)과 스택을 이용한 선형 시간 알고리즘: 19.3절 $O(n)$
- 상호 배타적 집합(*disjoint set, union-find*)(25장)을 이용: $O(n \log n)$
- 구간 트리(*segment tree*)(24장)를 이용: $O(n \log n)$



스위핑과 스택을 이용한 울타리 잘라내기 알고리즘: 19.3절

어떤 판자를 완전히 포함하는 사각형 중 면적이 최대인 사각형을
해당 판자(i 번 판자)의 최대 사각형이라 하자.



- 이 사각형의 **높이**는 i 번 판자와 항상 같다.
- 이 사각형의 **왼쪽 끝**과 **오른쪽 끝**은 i 번 판자보다 낮은 판자들로 막혀 있다. 이 판자들을 각각 $left[i]$, $right[i]$ 라 하자.
- i 번 판자가 맨 왼쪽이나 맨 오른쪽에 있을 수 있으므로, 주어진 판자들의 **양쪽 끝**에 **높이 0인 판자**를 붙여 두자.



$left[i], right[i]$ 를 쉽게(빠르게) 알 수 있으면?

$$\text{최대 사각형의 넓이} = (right[i] - left[i] - 1) \times h[i]$$

모든 판자에 대해 최대 사각형의 넓이를 구한 후, 그중 최댓값을 찾아 해결할 수 있다.

따라서, 울타리 잘라내기를 $O(n)$ 에 수행하려면

각 상자의 $left[i], right[i]$ 를 찾는 상수 시간($O(1)$) 알고리즘을 찾으면 된다.



라인 스위핑 알고리즘: *line sweeping algorithm*

주로 2차원 평면 기하 문제에서 많이 사용되는 알고리즘 디자인 패턴

평면 위에서 수평선(또는 수직선)을 ‘쓸고 지나가면서(*sweeping*)’ 문제를 해결하는 방식

예) 선분 교차 문제: 평면 위 선분들의 집합이 주어질 때, 이들 중 서로 교차하는 것이 있는가?

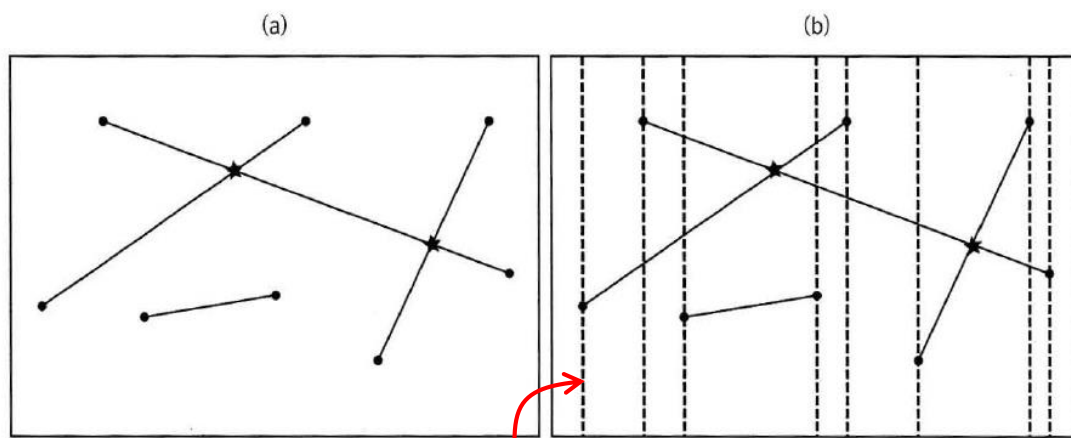


그림 15.19 평면의 선분들 간의 교차 여부를 구하는 샤모스-호이 알고리즘의 동작 원리

수직선

.....> *sweeping*

수직선이 선분의 끝점을 만나는 **이벤트**마다,
왼쪽 끝점을 만나면 선분 집합에 추가 (**교차 확인**)
오른쪽 끝점을 만나면 선분 집합에서 삭제
*Balanced BST*로 선분 집합을 관리하면?
삽입과 삭제가 모두 $O(\log n)$ 에 가능



울타리 잘라내기 문제를 위한 스위핑 알고리즘의 설계

스위핑: 맨 왼쪽 0번 판자부터 시작해서 끝까지 순서대로 각 판자를 처리

0번 판자: 이 판자의 왼쪽에는 아무 판자도 없다.

$left[0] = -1$: 높이가 0인 판자가 -1 위치에 있다고 가정

i 번 판자를 처리할 때:

왼쪽에 자신보다 높은 판자들이 남아 있다면, 그들의 최대 사각형을 i 번 판자가 막고 있음

왼쪽에 있는 판자들에 대해 최대 사각형의 넓이를 계산하고 지워 버린다.

그러면, i 번 판자 왼쪽에는 항상 자신보다 낮은 판자만이 남아 있으므로 $left[i]$ 를 바로 알 수 있다.



Lec. 03. 분할 정복

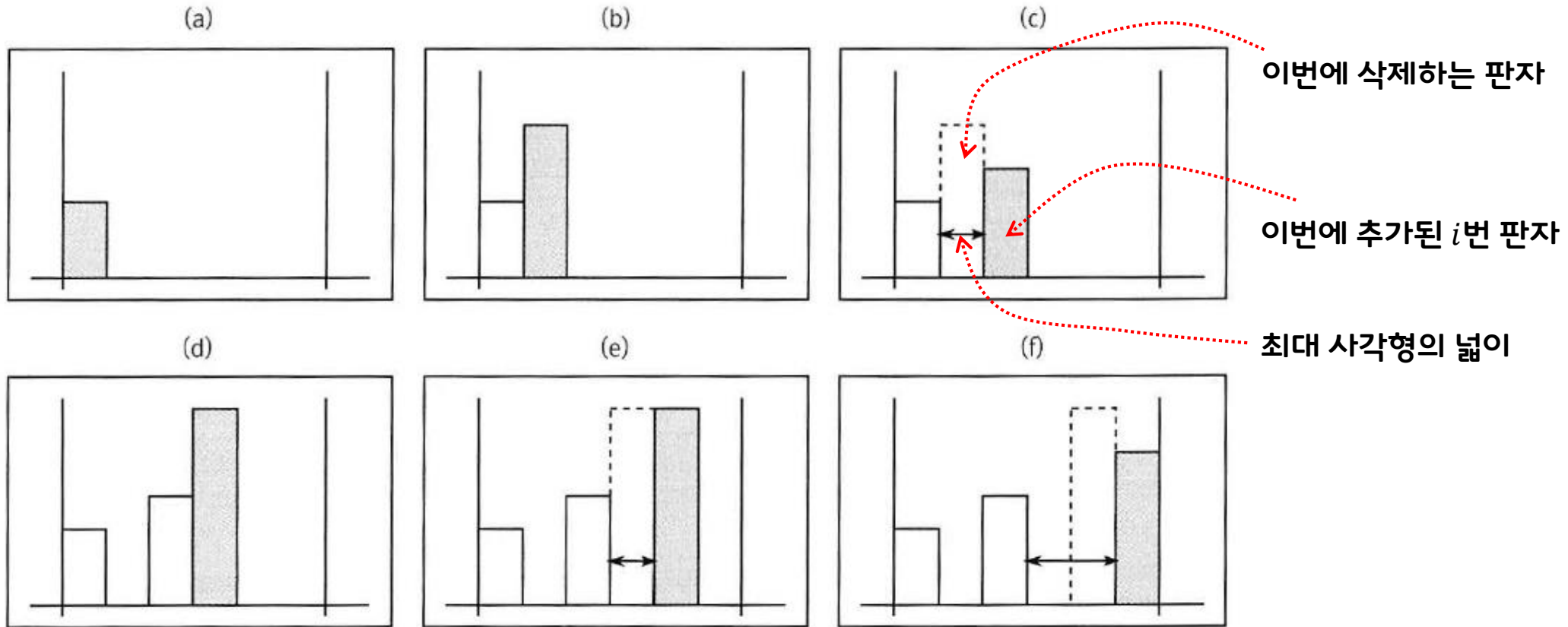


그림 19.3 울타리 자르기 문제를 푸는 스위핑 알고리즘의 동작 과정

새 판자를 만날 때마다 그 판자보다 높은 판자들을 전부 지우기 때문에 지원되지 않고 남은 사각형들의 높이는 갈수록 높아진다.



Lec. 03. 분할 정복

높이가 0인 가상의 판자

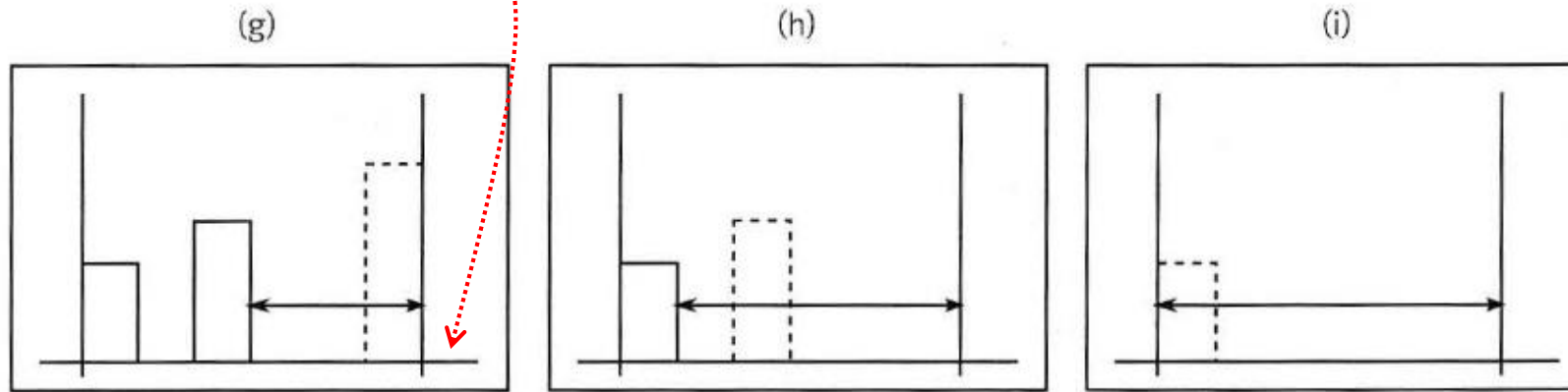


그림 19.3 울타리 자르기 문제를 푸는 스위핑 알고리즘의 동작 과정

맨 오른쪽에 있는 높이가 0인 가상의 판자를 만났을 때는
남아 있는 모든 판자들이 지워진다.



Lec. 03. 분할 정복

'''

P.3.2: 울타리 잘라내기 (ID: FENCE): 스위핑 + 스택 알고리즘

'''

```
for _ in range(int(input())):
    n = int(input())
    h = list(map(int, input().split()))
    print(fence(h))
```



Lec. 03. 분할 정복

```
def fence(h):  
    h.append(0) # 마지막에 높이가 0인 가상의 판자를 추가해 둔다.  
    stack = []  
    ret = 0  
    for i in range(len(h)):  
        # 남아 있는 판자들 중 오른쪽 끝 판자가 h[i]보다 높거나 같으면  
        # 이 판자의 최대 사각형은 i에서 끝난다.  
        while stack and h[stack[-1]] >= h[i]:  
            j = stack.pop()  
            # j번째 판자 왼쪽에 판자가 하나도 안 남아 있는 경우 left[j] = -1  
            # 아닌 경우, left[j] = 남아 있는 판 중 가장 오른쪽에 있는 판자의 번호  
            width = i if not stack else (i - stack[-1] - 1)  
            ret = max(ret, h[j] * width)  
        stack.append(i)  
    return ret
```




스위핑 + 스택 알고리즘의 시간 복잡도 분석

알고리즘이 이중 *for* 문으로 구현되었으므로, 시간 복잡도는 $O(n^2)$ 아닌가?

while 문 내부가 몇 번이나 반복되는가를 살펴보자.

while 문이 한 번 수행될 때마다 *stack*에서 원소가 하나씩 빠진다.

각 판자 인덱스는 스택에 정확히 한 번 push되고, 최대 한 번 pop된다.

따라서 *stack*에는 정확히 n 개의 원소가 들어가므로, *while*문의 수행 횟수는 $O(n)$ 이다.

Any Questions?

